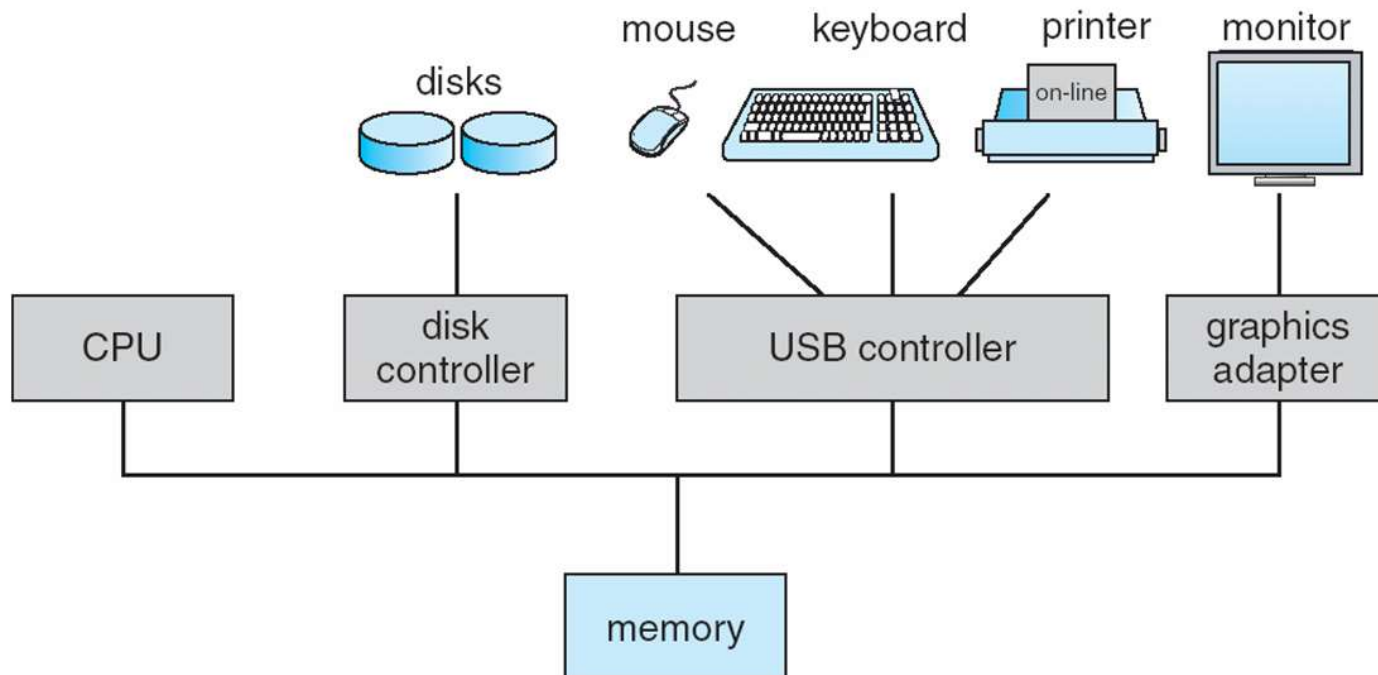


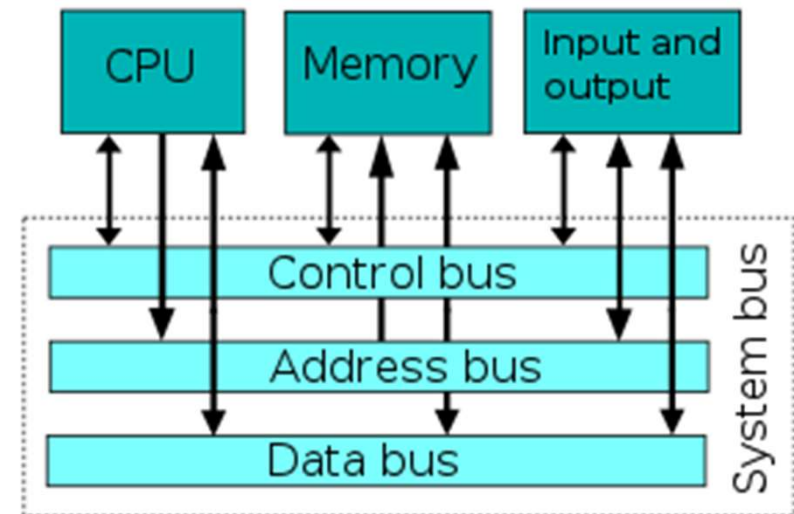
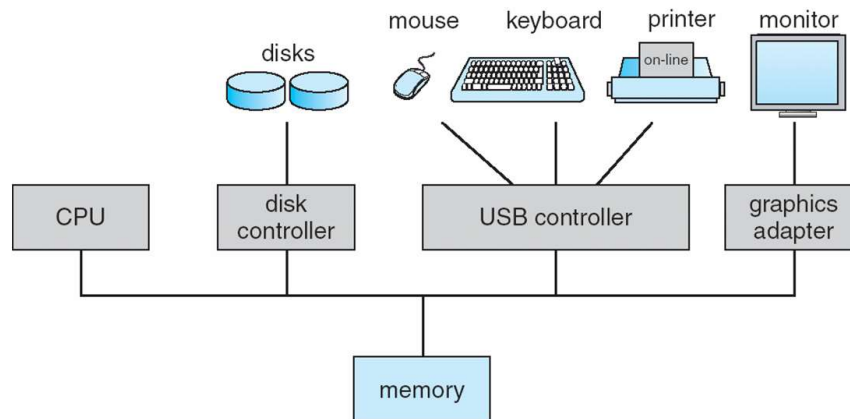
Organizzazione Calcolatore

- Il Sistema Operativo è strettamente legato all'architettura del calcolatore:
 - Una o più CPU ed i controllori di dispositivo (**driver**) sono connessi da un canale (**bus**) che permette l'accesso alla memoria condivisa
 - La CPU e i controllori possono eseguire operazioni in parallelo competendo per l'accesso alla memoria



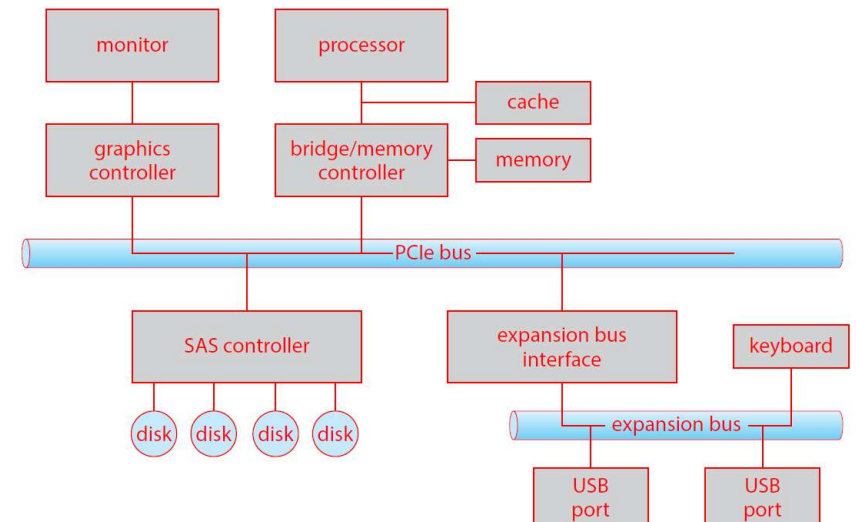
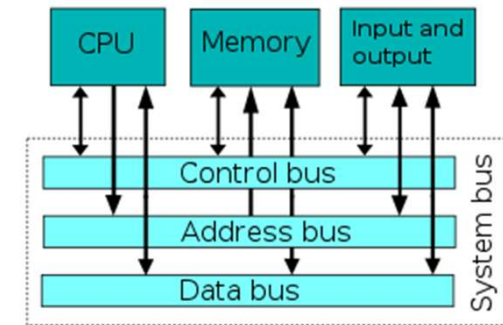
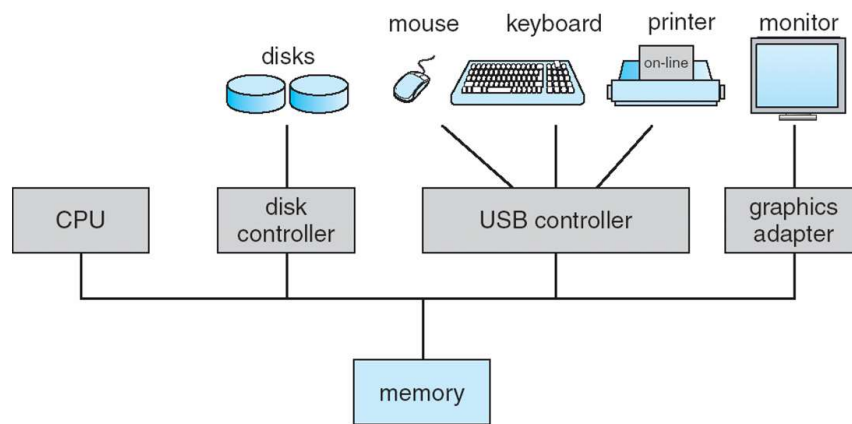
Organizzazione Calcolatore

- Il Sistema Operativo è strettamente legato all'architettura del calcolatore
 - Una o più CPU ed i controllori di dispositivo (**driver**) sono connessi da un canale (**bus**) che permette l'accesso alla memoria condivisa
 - La CPU e i controllori possono eseguire operazioni in parallelo competendo per l'accesso alla memoria



Organizzazione Calcolatore

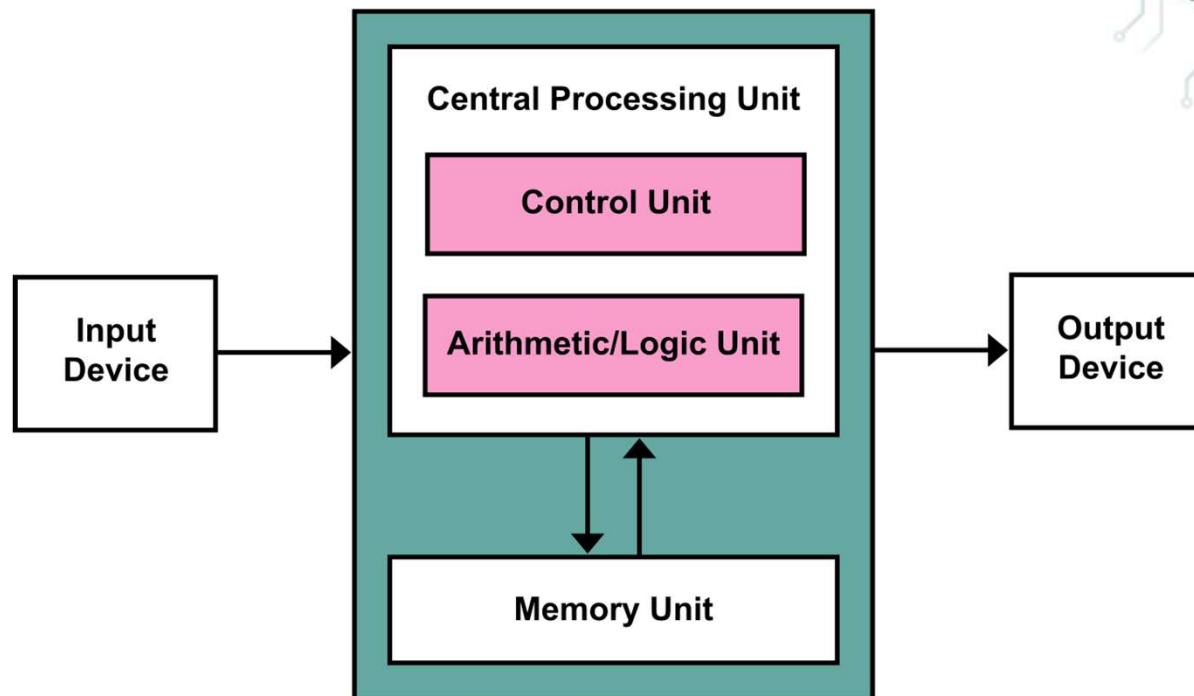
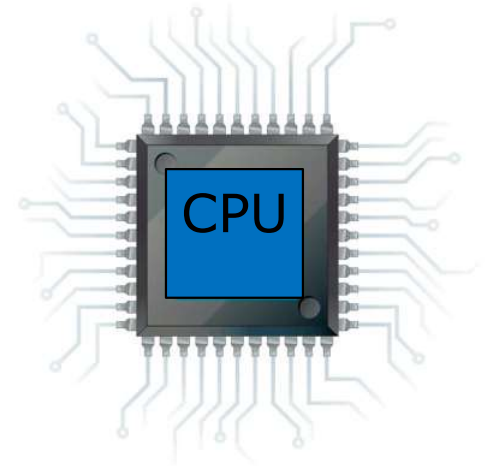
- Il Sistema Operativo è strettamente legato all'architettura del calcolatore
 - Una o più CPU ed i controllori di dispositivo (**driver**) sono connessi da un canale (**bus**) che permette l'accesso alla memoria condivisa
 - La CPU e i controllori possono eseguire operazioni in parallelo competendo per l'accesso alla memoria



In realtà più bus (PCIe, PCI, DMI, etc.) interconnessi, il principale è PCIe (Peripheral Component Interconnect Express)

CPU

- Central Processing Unit
 - Esegue le istruzioni dalla memoria:
 - ▶ Ogni CPU ha un **set di istruzioni** che può eseguire
 - ▶ **Ciclo**: fetch, decode, execute

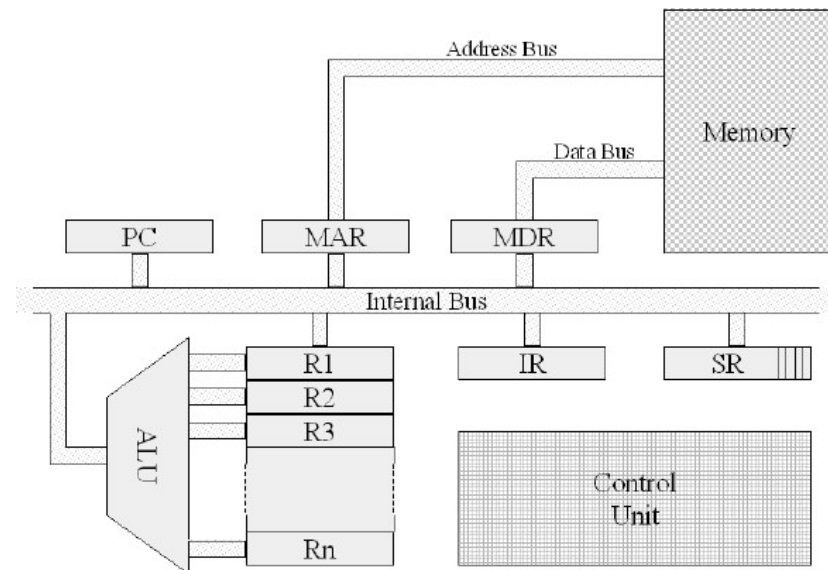
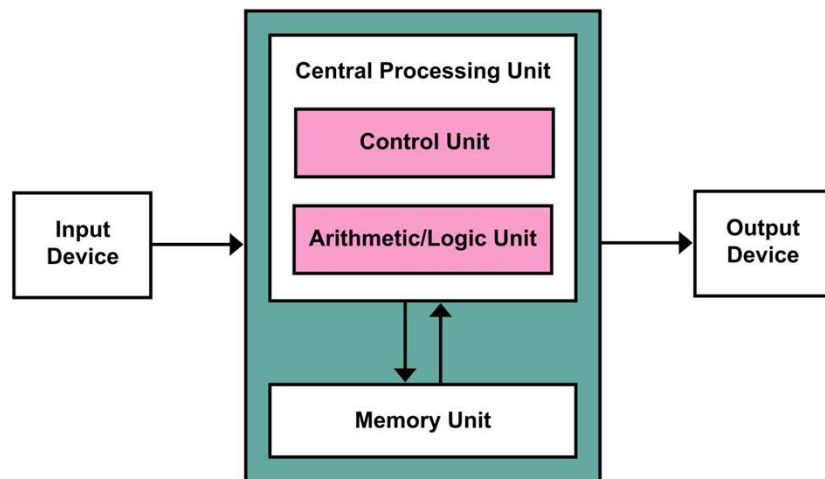
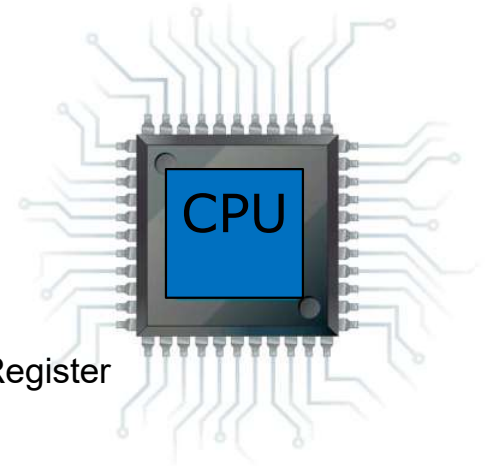


CPU

□ Central Processing Unit

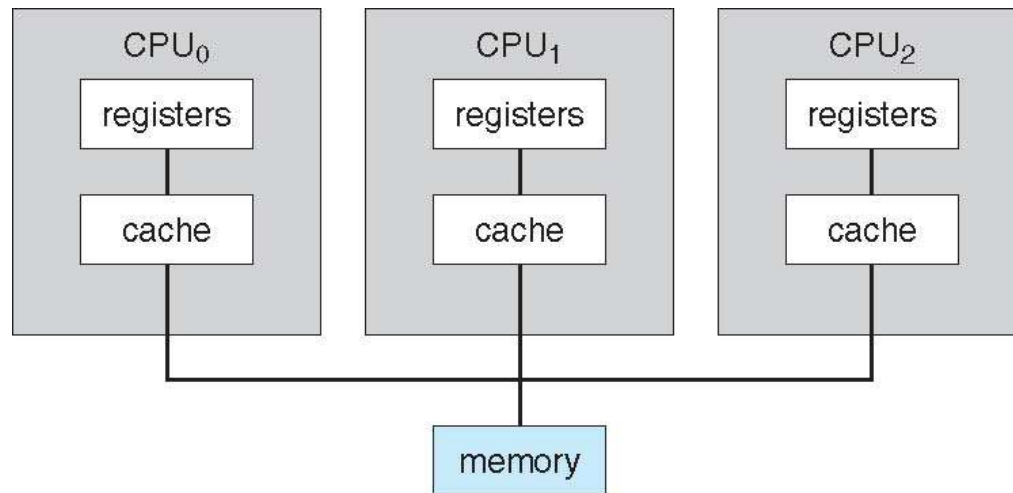
□ Esegue le istruzioni dalla memoria:

- ▶ Ogni CPU ha un **set di istruzioni** che può eseguire
- ▶ **Ciclo**: fetch, decode, execute
- ▶ **Registri**:
 - Program Counter, Stack Pointer, Instruction Register, Status Register
Memory Add. Reg., Memory Data Reg.



CPU

- **CPU**: componente hardware che esegue le istruzioni
- **Processore**: un chip fisico che contiene una o più CPU
- **Unità di calcolo** (*core*): unità di elaborazione di base della CPU
- **Multicore**: che include più unità di calcolo sulla stessa CPU
- **Multiprocessore**: che include più processori



Multiprocessori simmetrici

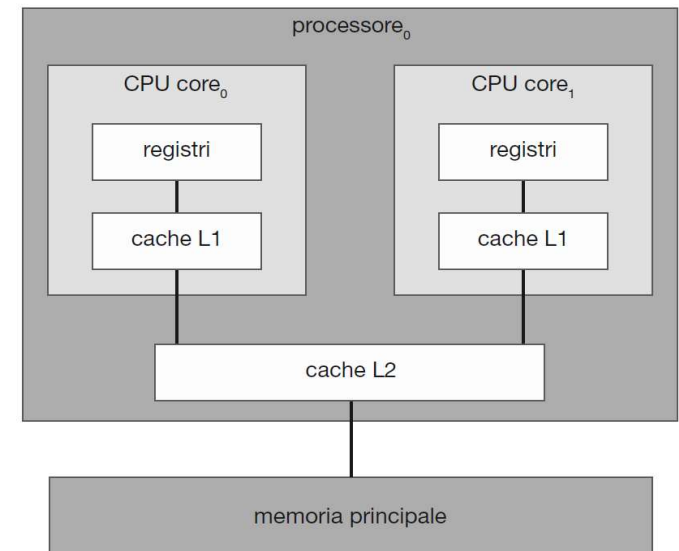
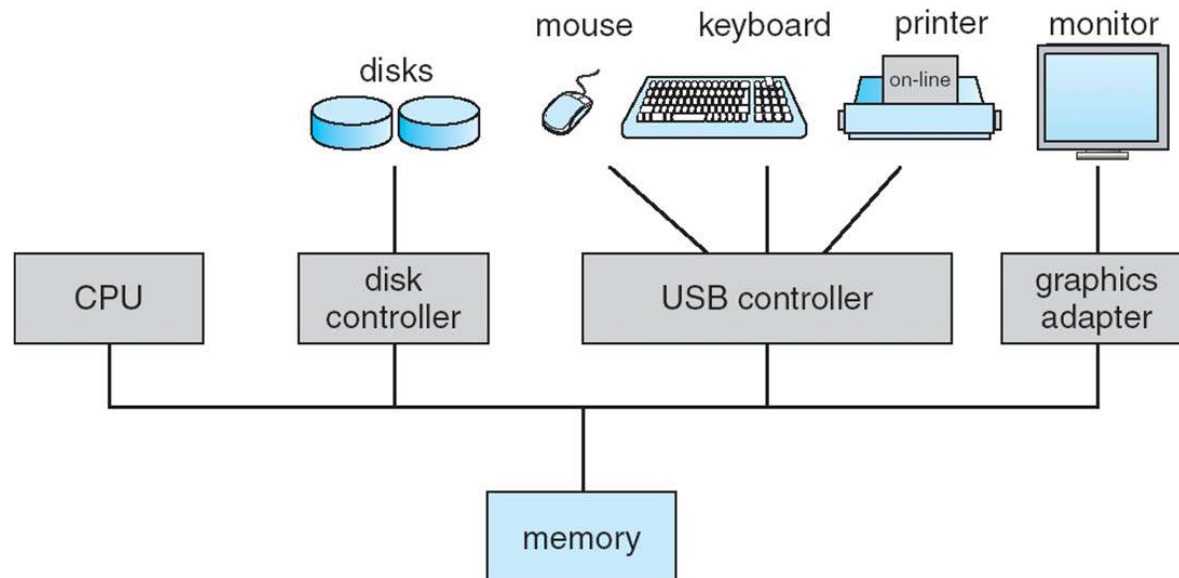


Figura 1.9 Architettura dual-core, con due unità sullo stesso chip.

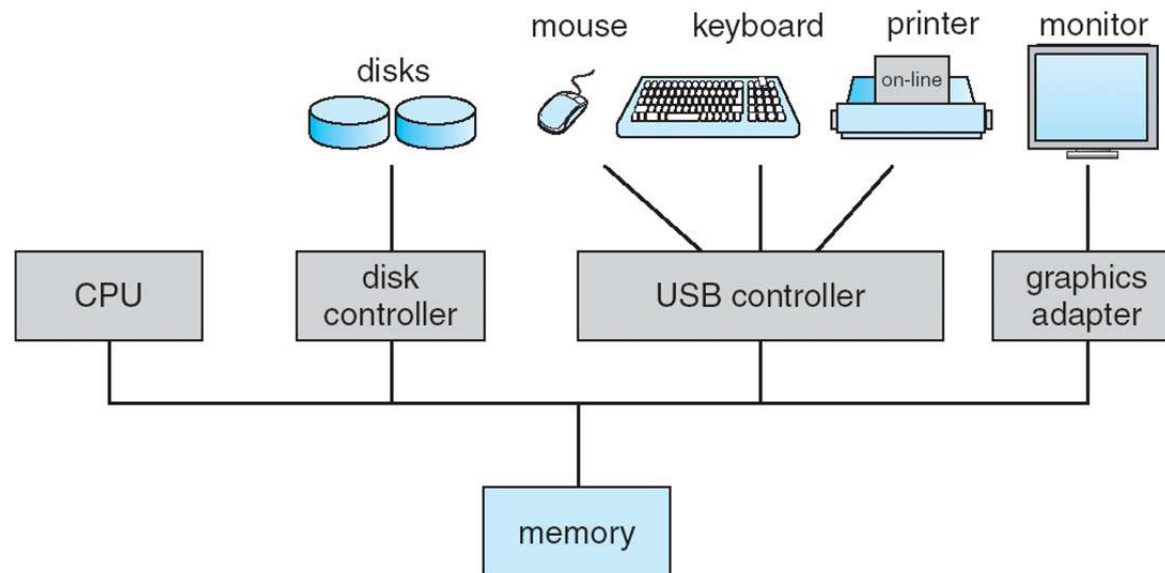
Sistemi con Interrupt

- ❑ I dispositivi I/O e la CPU sono in esecuzione concorrente
- ❑ Ogni **controllore di dispositivo** è responsabile di un tipo di dispositivo
- ❑ Ogni controllore ha un **buffer locale**
- ❑ CPU porta dati dalla/alla memoria ai/dai buffers locali
- ❑ I/O dal dispositivo ai buffer locali del controller
- ❑ Ogni controllore informa la CPU che ha finito le sue operazioni attraverso il meccanismo delle interruzioni (**interrupt**)



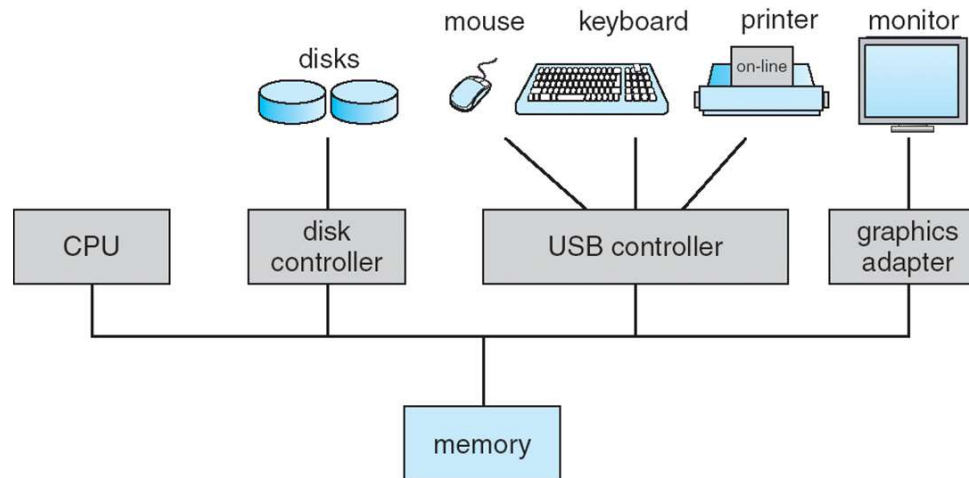
Interrupt vs Polling

- **Interrupt e polling** sono due modalità con cui gli eventi generati dai dispositivi I/O possono essere gestiti dalla CPU
 - Con il **polling**, la CPU tiene traccia delle comunicazioni dei dispositivi di I/O interrogandoli ad intervalli regolari
 - Con **interrupt**, il dispositivo di I/O interrompe la CPU comunicando ad essa che ha bisogno di andare in esecuzione



Interruzioni

- Le interruzioni permettono di gestire le operazioni concorrenti
 - Sovrapporre CPU e operazioni I/O
 - Evitare **busy waiting**



- Un **Sistema Operativo moderno è guidato dalle interruzioni**
 - Le interruzioni trasferiscono il controllo ad una procedura di servizio dell'interruzione
 - Una volta eseguita la routine di servizio il controllo ritorna all'operazione interrotta

Interrupt Timeline

- ❑ Per avviare I/O la CPU carica opportuni registri del controllore I/O
- ❑ La CPU continua il processamento ...
- ❑ intanto il controllore I/O avvia le operazioni sul proprio buffer ...
- ❑ ... quando ha terminato invia l'**interrupt**

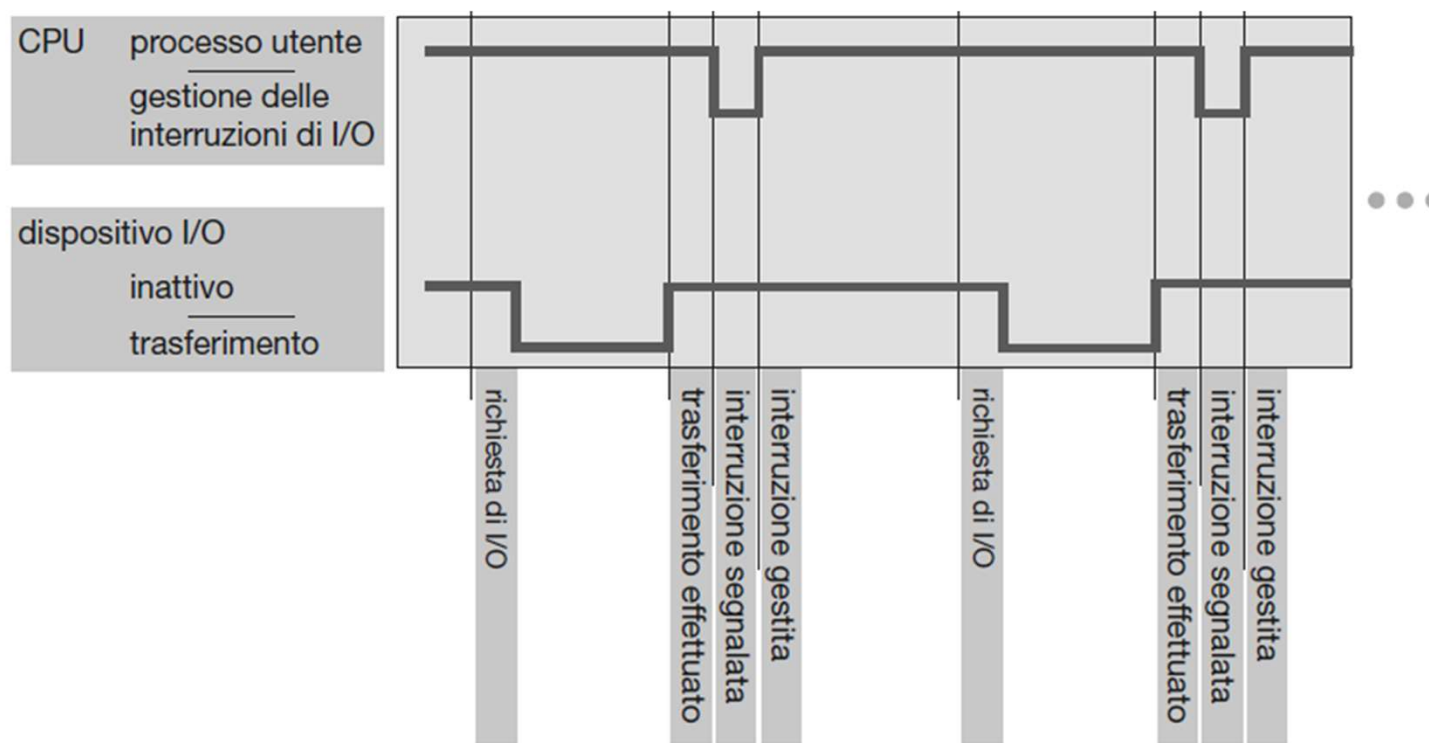


Figura 1.3 Diagramma temporale delle interruzioni per un singolo programma che invia dati in output.

Interrupt Timeline

- ... la CPU controlla l'interruzione periodicamente
- Determina il tipo di interruzione e gestisce l'interruzione
 - **Gestore interruzione** invoca la routine di servizio
- Gestita l'interruzione la CPU riprende l'elaborazione interrotta

Fetch();

Decode();

Execute();

CheckForInterrupt();

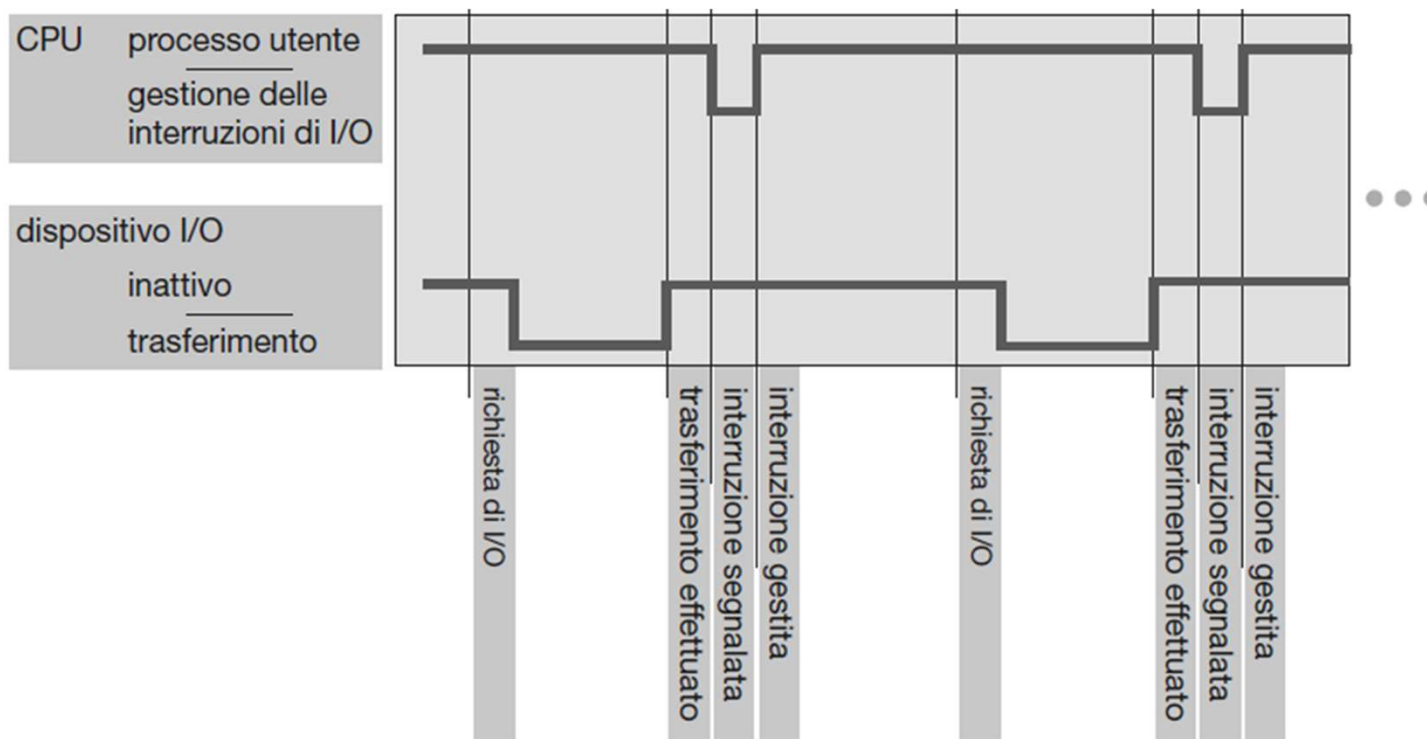


Figura 1.3 Diagramma temporale delle interruzioni per un singolo programma che invia dati in output.

Gestione delle Interruzioni

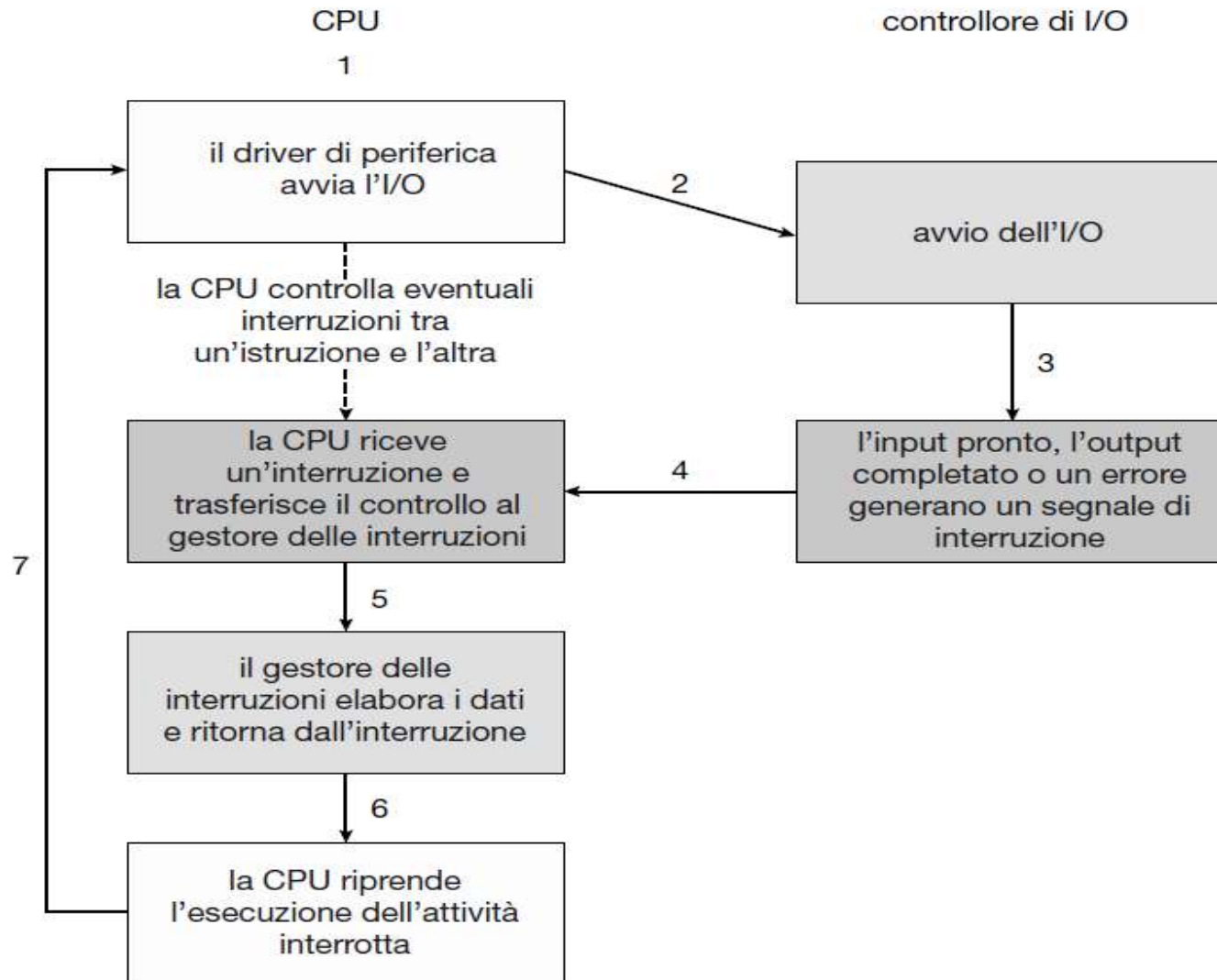
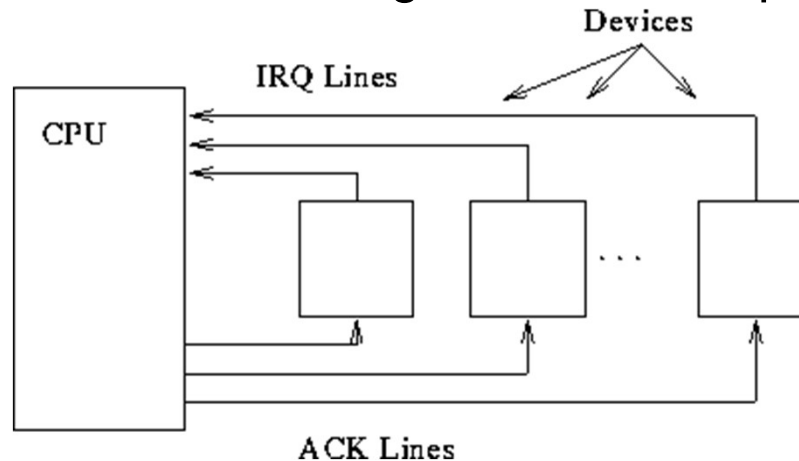


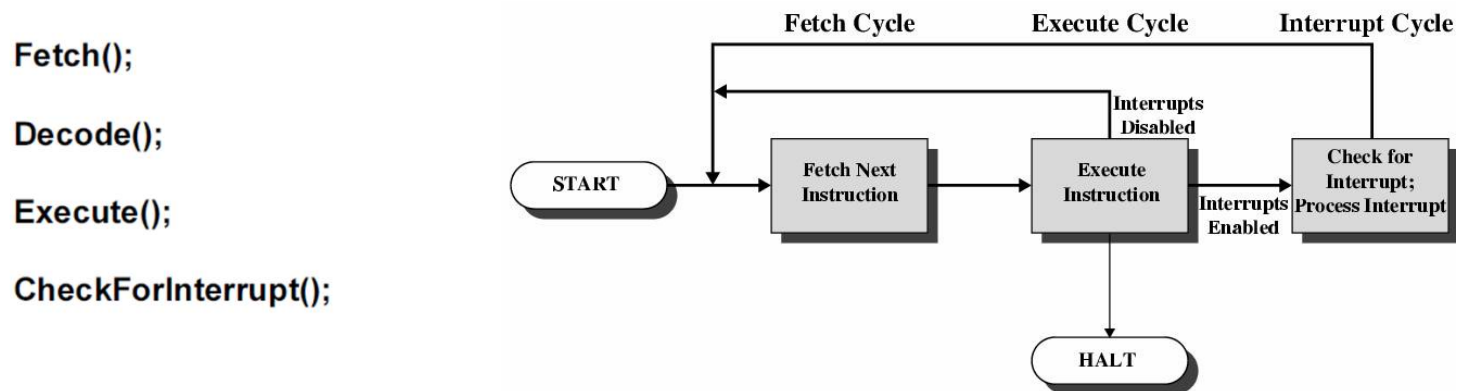
Figura 1.4 Ciclo di I/O guidato dalle interruzioni.

Gestione delle Interruzioni

- Il dispositivo I/O invia un segnale su Interrupt Request Line (IRQ)



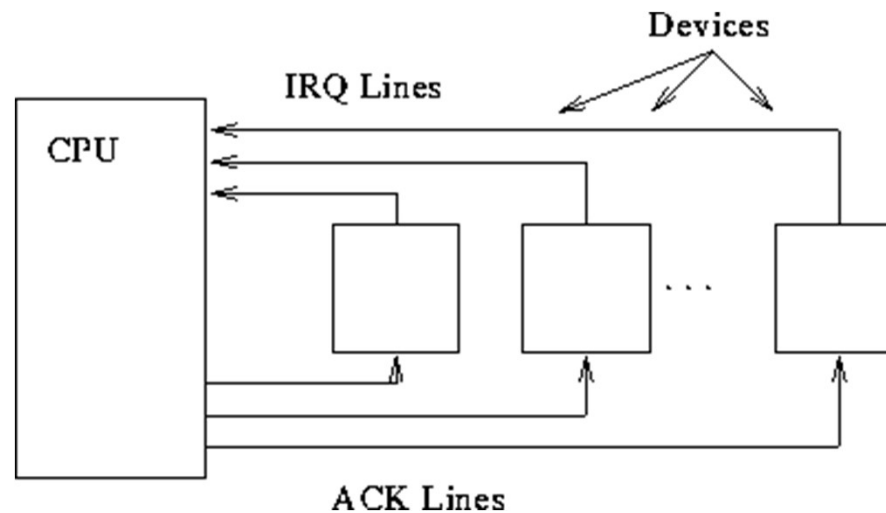
- La CPU controlla la IRQ per ogni istruzione ...



- Due tipi di interruzione (**nonmasksable/maskable**)
 - Non mascherabile (errori irreversibili) – int. disable
 - Macherabile (utilizzata dai controllori di dispositivo) – int. enable

Gestione delle Interruzioni

- Gestione interruzioni:
 - Il dispositivo genera il segnale (IRQ)
 - Il processore riceve il segnale e interrompe il programma in esecuzione
 - Il processore informa il dispositivo che l'interrupt è stato ricevuto (ACK)
 - Viene eseguita la **procedura di servizio** (ISR)
 - Si ripristina il processo interrotto



Gestione delle Interruzioni

- Il dispositivo I/O invia un segnale su **Interrupt Request Line (IRQ)**
- La CPU controlla la **IRQ** per ogni istruzione ...
 - Fetch();
 - Decode();
 - Execute();
 - CheckForInterrupt();
- Per servire la richiesta la CPU:
 - **Salva lo stato** dell'operazione interrotta
 - ▶ minimo contesto: registri e program counter
 - **Trova l'indirizzo** della procedura di servizio (**interrupt handler**) ...
 - **Passa il controllo** alla procedura di servizio fornendone l'indirizzo iniziale
 - ▶ La procedura di servizio viene eseguita (salva il resto dello stato modificato)
 - ▶ La procedura di servizio esce (**interrupt handler exit**)
 - Il **controllo torna** all'operazione interrotta (recuperando lo stato esecutivo)

Gestione delle Interruzioni

- Per servire la richiesta la CPU:
 - Riceve l'interruzione
 - Determina il tipo di interruzione
 - Trova l'indirizzo della procedura di servizio (**interrupt handler**) ...
 - Passa il controllo alla **procedura di servizio** fornendone l'indirizzo iniziale

- **Interrupt vettorizzato**
 - ▶ Il dispositivo manda un codice di interruzione
 - ▶ Il codice è un indice nel **vettore delle interruzioni** che contiene l'indirizzo della proc. servizio
 - ▶ Il **gestore dell'interruzione** salva lo stato dell'operazione interrotta (registri e program counter)
 - ▶ La **procedura di servizio** viene eseguita:
 - ▶ Il controllo torna all'operazione interrotta (recuperando lo stato esecutivo)

Gestione delle Interruzioni

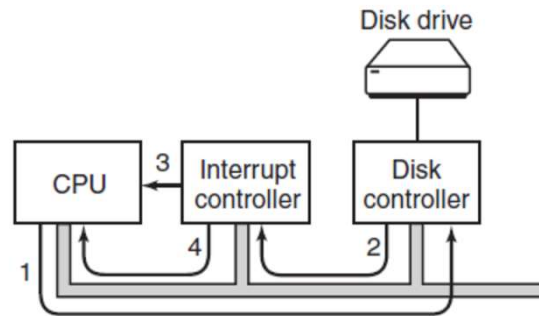
- Vettore delle interruzioni dei processori Intel, da 0 a 31 **non mascherabili**, da 32 a 255 **mascherabili**

numero di vettore	descrizione
0	errore di divisione
1	eccezione di debug
2	interruzione null
3	breakpoint
4	eccezione di overflow
5	eccezione di range exceeded
6	codice operativo non valido
7	dispositivo non disponibile
8	doppio errore
9	overrun del segmento coprocessore (riservato)
10	task state segment (tss) non valido
11	segmento non presente
12	errore di stack
13	protezione generale
14	errore di pagina
15	(riservato Intel, non utilizzare)
16	errore in virgola mobile
17	controllo dell'allineamento
18	controllo della macchina
19–31	(riservato Intel, non utilizzare)
32–255	interruzioni mascherabili

Figura 1.5 Tabella degli eventi di un processore Intel.

Gestione delle Interruzioni

- Nei SO moderni il **controllore hardware di interruzione** fornisce meccanismi di interruzione sofisticati:
 - Posticipare la gestione dell'interruzione durante fasi critiche
 - Gestione multi-livello delle priorità (alta priorità, bassa priorità)
 - Gestione efficiente delle routine di servizio



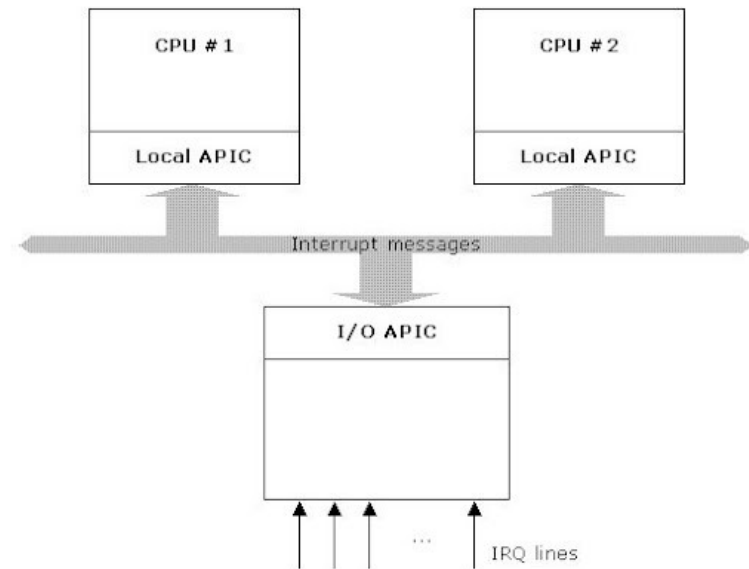
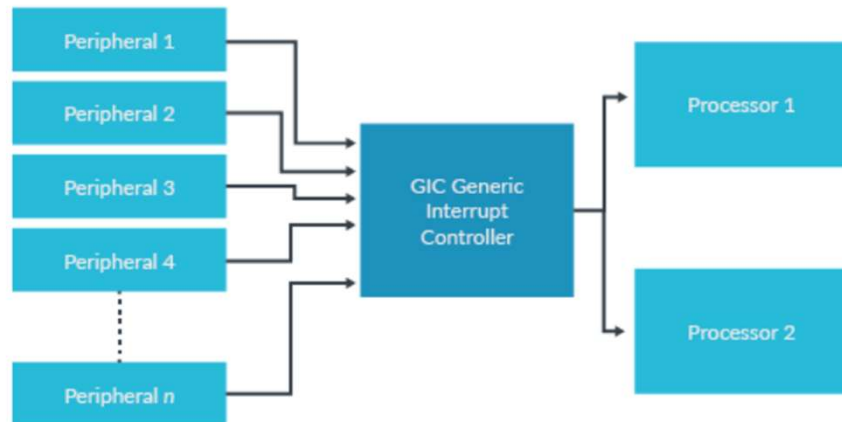
- Due tipi di interruzione (**nonmasksable/maskable**) su due linee di interrupt
 - ▶ Non mascherabile (errori irreversibili)
 - ▶ Mascherabile (utilizzata dai controllori di dispositivo), può essere spenta dalla CPU durante le operazioni critiche
- Di solito più servizi che elementi nell'interrupt vector
 - ▶ **Interrupt chaining** (numero di interrupt punta ad una lista di interrupt handler)

Gestione delle Interruzioni

- Nei SO moderni il **controllore hardware di interruzione** fornisce meccanismi di interruzione sofisticati:
 - Posticipare la gestione dell'interruzione durante fasi critiche
 - Gestione multi-livello (alta priorità, bassa priorità)
 - Modo efficiente per lanciare il corretto gestore per un dispositivo

ARM – (GIC Generic Interrupt Controller)

x86 – (APIC Advanced Programmable Interrupt Controller)

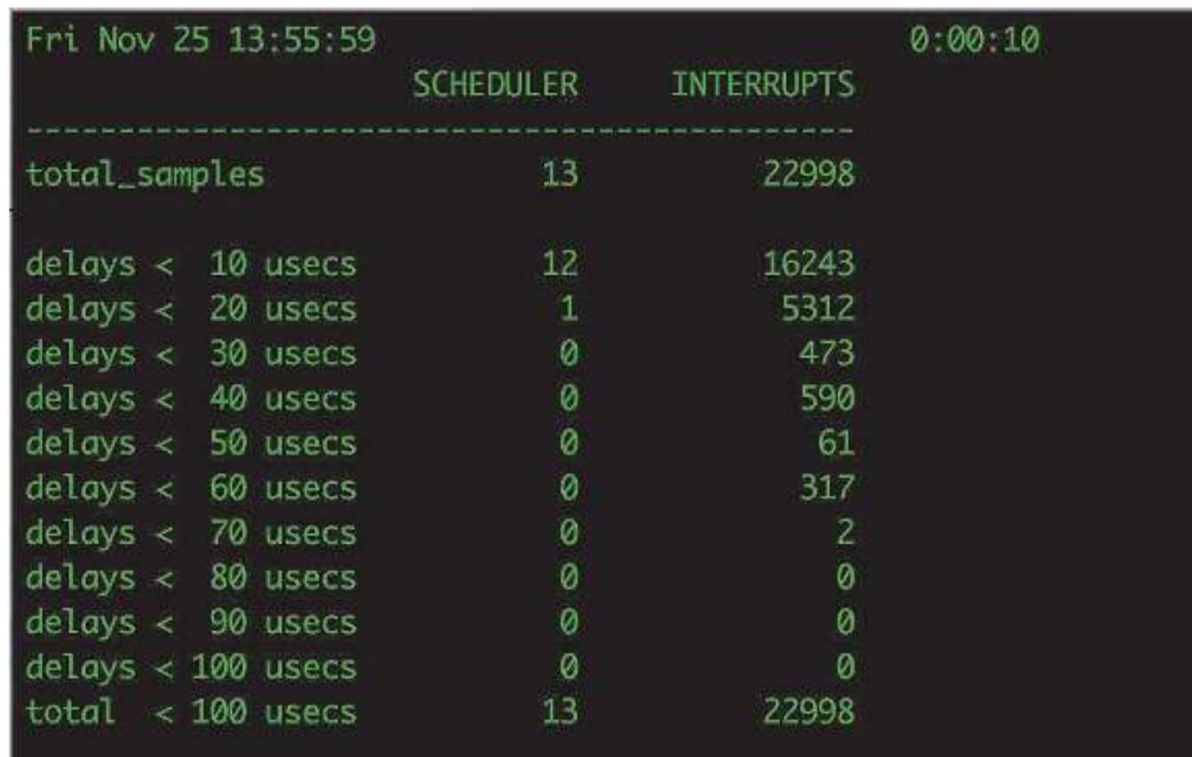


Sistemi Operativi e Interruzioni

- I Sistemi Operativi sono **interrupt driven** (hardware e software)
 - Hardware interrupt da un dispositivo
 - Software interrupt (**exception** o **trap**):
 - ▶ Errori Software (e.g., division by zero)
 - ▶ Operazioni non lecite
 - ▶ Gestione dei processi o della memoria
 - ▶ **Richiesta per un servizio** del sistema operativo
 - ▶ Altri problematiche ...

Sistemi Operativi e Interruzioni

- Importante gestione attenta degli interrupt
- Anche un PC quasi idle gestisce continuamente interrupt
- In figura comando latency macOS su desktop
 - ... in 10 secondi 23000 interrupts



The screenshot shows the output of the 'latency' command in macOS, displaying statistics for the scheduler and interrupts over a 10-second period. The output is formatted as a table with two main columns: SCHEDULER and INTERRUPTS. The data shows that the scheduler processed 13 samples, while the interrupt handler processed 22,998 samples. The majority of interrupts (16,243) occurred within the first 10 microseconds, with the number of interrupts decreasing as the delay threshold increases.

	SCHEDULER	INTERRUPTS
Fri Nov 25 13:55:59		0:00:10

total_samples	13	22998
delays < 10 usecs	12	16243
delays < 20 usecs	1	5312
delays < 30 usecs	0	473
delays < 40 usecs	0	590
delays < 50 usecs	0	61
delays < 60 usecs	0	317
delays < 70 usecs	0	2
delays < 80 usecs	0	0
delays < 90 usecs	0	0
delays < 100 usecs	0	0
total < 100 usecs	13	22998

Caratteristiche Sistema Operativo

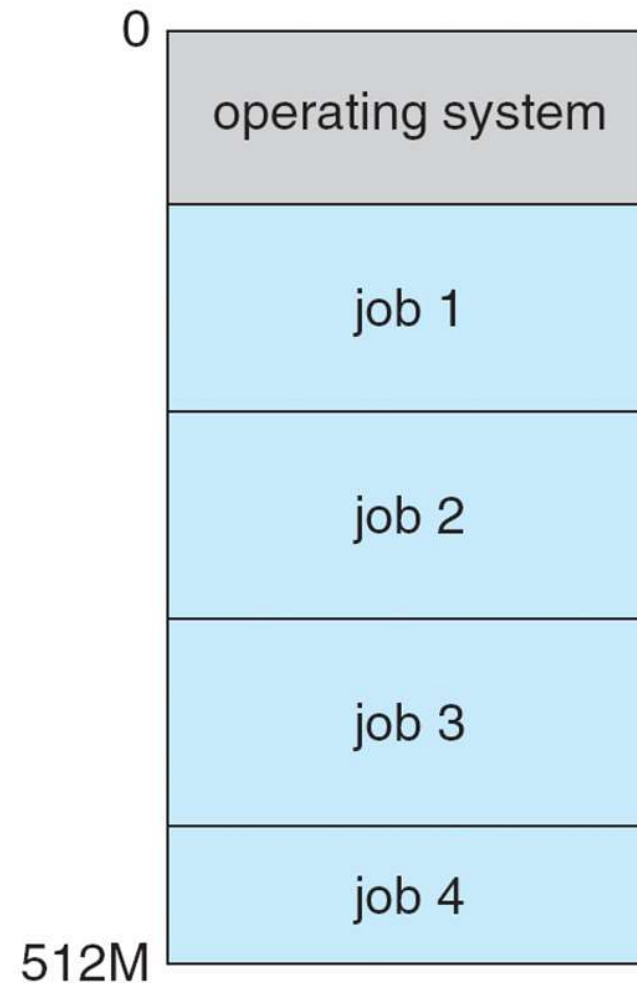
- **Multiprogrammazione** per efficienza
 - Un singolo utente non riesce ad utilizzare al meglio CPU e dispositivi I/O
 - Multiprogrammazione organizza i jobs (codice e dati) per mantenere sempre attiva la CPU
 - Un sottinsieme dei job totali del sistema è mantenuto in memoria
 - Un job viene selezionato ed eseguito dal **job scheduler**
 - Nel caso di attesa (operazioni I/O per esempio) il SO cambia il job

- **Timesharing (multitasking)** la CPU che cambia job molto frequentemente permettendo agli utenti di interagire (**interactive** computing)
 - **Tempo di risposta** breve
 - Ogni utente ha programmi in esecuzione ⇒ **processi**
 - Se più jobs sono pronti per essere eseguiti ⇒ **CPU scheduling**

Attività del Sistema Operativo

1. **Programma di avvio** (*bootstrap program*).
2. **Il kernel** inizia a offrire servizi al sistema e agli utenti.
3. **Interruzioni ed eccezioni** (*traps o exceptions*).
4. **Chiamata di sistema o system call**.
5. **Multiprogrammazione** → *multitasking*.

Memoria per Sistema a Multiprogrammazione



Modalità Operativa Duale

- ❑ Sistemi multiprogrammati e multiutente richiedono **meccanismi di protezione**
- ❑ Operazioni in **dual-mode**
 - ❑ **User mode** e **kernel mode** (modo utente e supervisore)
 - ❑ Permettono all'OS di proteggersi e proteggere altri componenti

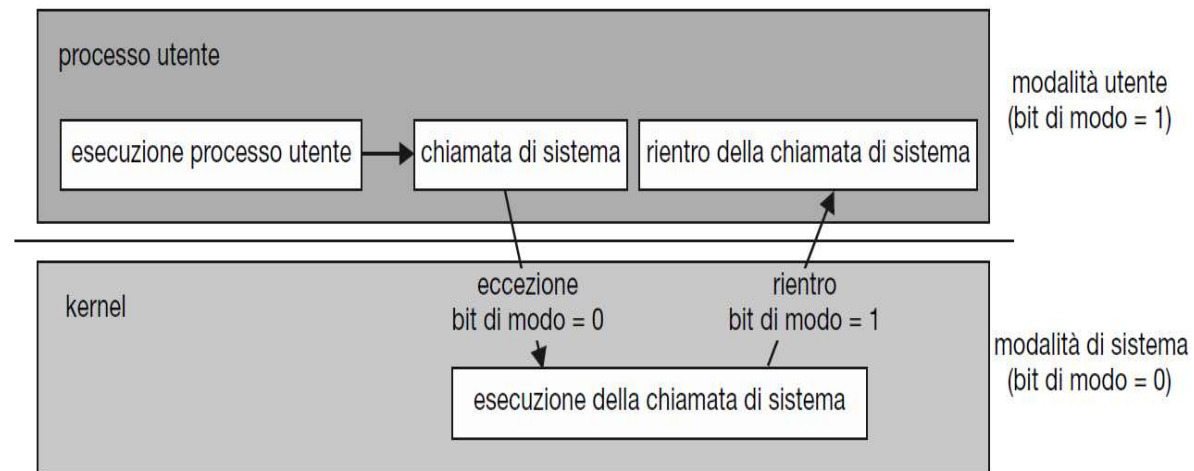


Figura 1.13 Transizione da modalità utente a modalità di sistema.

- ❑ Un **bit di modalità** (hardware)
 - ▶ Distinguere quando il sistema esegue codice utente o kernel
 - ▶ Alcune istruzioni sono **privilegiate** quindi solo eseguibili in modalità kernel
 - ▶ Interrupt/trap modalità kernel, servita l'interruzione il SO ripristina la modalità user
 - MS-DOS per Intel 8088 non aveva bit di modo

Protezione CPU - tempo

- Programmi utenti non devono mantenere il controllo indefinito
 - **Limite temporale**
- Timer per dare a SO controllo periodico della CPU
 - Timer invia segnale d'interruzione alla CPU dopo una latenza fissata
 - ▶ Contatore decrementato dal clock fisico
 - SO setta il contatore (modalità privilegiata) a zero genera un interrupt
 - **Permette ad SO di riprendere il controllo o terminare processi**
- Il tempo macchina della CPU è quantizzato in ticks
- Ogni processo viene eseguito per un certo numero di ticks e poi messo in stato di attesa.